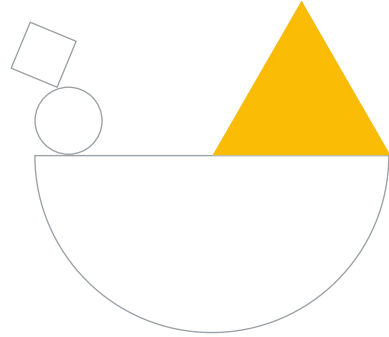


인프라 자동화



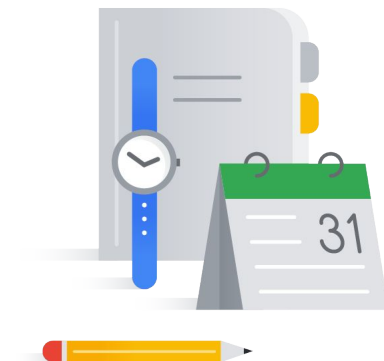
지금까지 Google Cloud의 몇 가지 서비스와 기능을 살펴보았습니다. 이번에는 Google Cloud 인프라의 배포를 자동화하는 방법을 알아보겠습니다.

코드에서 Cloud API를 호출하는 것은 인프라를 생성하는 효과적인 방법입니다. 그러나 코드를 작성하여 인프라를 만드는 데는 몇 가지 어려움이 있습니다. 그중 하나로 인프라의 유지관리성이 소프트웨어의 품질에 직접적인 영향을 받는다는 문제를 들 수 있습니다. 예를 들어 하나의 프로그램에 VM을 만들기 위한 Cloud API 호출이 수십 개가 있을 수 있는데, VM 하나의 정의에서 발생한 문제를 해결하려면 먼저 수십 개의 호출 중에서 이 문제를 유발한 호출이 무엇인지 알아내야 합니다. 표준적인 소프트웨어 개발 권장사항도 적용되고, 애플리케이션에 빠른 속도로 변경이 적용되기 때문에 코드를 유지보수해야 합니다.

이에 따라 또 다른 수준의 구성이 필요합니다. Terraform은 이 용도로 사용됩니다. Terraform은 고도로 구조화된 템플릿 및 구성 파일로 이루어진 시스템을 사용하여 인프라를 가독성이 뛰어나고 이해하기 쉬운 형식으로 문서화합니다. Terraform은 실제 Cloud API 호출을 가려주기 때문에 코드를 작성할 필요가 없고 인프라의 정의에 집중할 수 있습니다.

주제

- 01 Terraform
실습: Terraform을 사용하여 인프라 배포 자동화
- 02 Google Cloud Marketplace



이 모듈에서는 **Terraform**을 사용하여 인프라의 배포를 자동화하는 방법과 **Google Cloud Marketplace**를 사용하여 인프라 솔루션을 출시하는 방법을 알아봅니다.

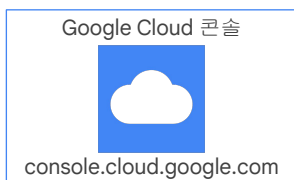
이 모듈의 실습에서는 **Terraform**을 사용하여 **VPC** 네트워크, 방화벽 규칙, **VM** 인스턴스를 배포해 봅니다.

먼저 **Terraform**에 관해 알아보겠습니다.



Terraform

Google Cloud 콘솔, Cloud SDK, Cloud Shell



☐ Name ^	Zone	Internal IP	External IP	Connect
☐ nginxstack-1	us-central1-f	10.128.0.3 (nic0)	35.238.84.245	SSH ▾ ⋮
☐ nginxstack-2	us-central1-f	10.128.0.4 (nic0)	35.225.177.18	SSH ▾ ⋮
☐ nginxstack-3	us-central1-f	10.128.0.2 (nic0)	35.239.250.238	SSH ▾ ⋮

Cloud Shell

Google Cloud SDK

```
$ gcloud compute instances list
```

NAME	ZONE	INTERNAL_IP	EXTERNAL_IP
nginxstack-1	us-central1-f	10.128.0.3	35.238.84.245
nginxstack-2	us-central1-f	10.128.0.4	35.225.177.18
nginxstack-3	us-central1-f	10.128.0.2	35.239.250.238

Google Cloud

지금까지는 Google Cloud 콘솔과 Cloud Shell을 사용하여 Google Cloud 리소스를 만들었습니다. 서비스를 처음 사용하거나 UI를 선호하는 사용자는 콘솔을 사용하는 것이 좋습니다. 특정 서비스를 익숙하게 사용할 줄 알고 명령줄을 통해 빠르게 리소스를 만들고 싶다면 Cloud Shell을 사용하는 것이 좋습니다. Terraform은 이러한 작업을 한 단계 더 발전시킵니다.

인프라의 빠른 프로비저닝 및 제거를 지원하는 코드형 인프라 (IaC)

- 필요할 때 인프라를 빌드할 수 있습니다.
- 사용하지 않는 인프라를 폐기할 수 있습니다.
- 개발, 테스트, 프로덕션용으로 동일한 인프라를 만들 수 있습니다.
- CI/CD 파이프라인의 일부가 될 수 있습니다.
- 템플릿은 재해 복구 절차의 빌딩 블록이 됩니다.
- 리소스 종속 항목과 복잡성을 관리할 수 있습니다.
- Google Cloud는 여러 IaC 도구를 지원합니다.



Google Cloud

Terraform은 코드형 인프라 (**IaC**)로 사용되는 도구 중 하나입니다. 본격적으로 **Terraform**을 살펴보기 전에 먼저 코드형 인프라가 어떤 일을 하는지 알아보겠습니다. 기본적으로 코드형 인프라는 인프라의 빠른 프로비저닝 및 삭제를 지원합니다.

배포를 주문형으로 프로비저닝하는 기능은 무척 강력합니다. 이는 지속적 통합 파이프라인에 통합되어 지속적 배포를 매끄럽게 지원할 수 있습니다.

인프라 프로비저닝이 자동화된다는 것은 인프라를 주문형으로 프로비저닝할 수 있고 배포 복잡성이 코드에서 관리됨을 의미합니다. 따라서 요구사항이 변경됨에 따라 인프라를 유연하게 변경할 수 있습니다. 또한 모든 변경사항을 한곳에서 관리할 수 있습니다. 개발, 테스트와 같은 환경의 인프라는 쉽게 프로덕션을 복제할 수 있고 사용하지 않을 때는 즉시 삭제할 수 있습니다. 이 모든 것이 코드형 인프라의 특성입니다.

IaC에 사용할 수 있는 도구에는 몇 가지가 있습니다. **Google Cloud**는 **Terraform**을 지원합니다. **Terraform**에서는 배포가 구성이라는 파일에 기술되고, 프로비저닝이 필요한 모든 리소스가 상세히 설명됩니다. 구성은 템플릿을 사용하여 모듈화할 수 있으며, 이를 통해 리소스를 여러 배포에서 재사용 가능한 구성요소로 추상화할 수 있습니다.

Google Cloud는 **Terraform** 외에도 다음과 같은 **IaC** 도구를 지원합니다.

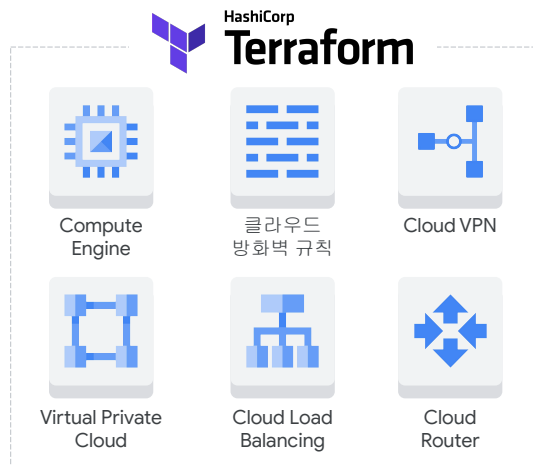
- Chef
- Puppet

- Ansible
- Packer

이 과정에서는 **Terraform**을 집중적으로 살펴봅니다.

인프라 자동화 도구 Terraform

- 반복적인 배포 프로세스
- 선언적 언어
- 애플리케이션 중심
- 동시 배포
- 템플릿 기반



Google Cloud

Terraform은 Google Cloud 리소스를 프로비저닝할 수 있는 오픈소스 도구입니다.

Terraform으로 선언적 구성 파일을 사용하여 가상 머신, 컨테이너, 스토리지, 네트워킹 등의 Google Cloud 리소스를 프로비저닝할 수 있습니다. 사용자는 애플리케이션에 필요한 모든 리소스를 선언적 형식으로 지정하고 구성을 배포하기만 하면 됩니다. HashiCorp 구성 언어(HCL)로 블록, 인수, 표현식을 사용하여 리소스를 간결하게 기술할 수 있습니다.

이 배포는 여러 차례 반복해도 동일한 결과를 얻을 수 있으며, 명령어 하나 또는 클릭 한 번으로 배포 전체를 삭제할 수 있습니다. 선언적 방식의 장점은 사용자가 구성을 지정하면 시스템이 어떤 조치를 취해야 하는지 알아낸다는 것입니다.

사용자는 각 리소스를 개별적으로 배포하는 대신 애플리케이션 또는 서비스를 구성하는 리소스 세트를 지정하면 되므로 애플리케이션에 집중할 수 있습니다. Terraform은 Cloud Shell과 달리 리소스를 동시에 배포합니다.

Terraform은 각 Google Cloud 서비스의 기본 API를 사용하여 리소스를 배포합니다. 따라서 사용자는 인스턴스, 인스턴스 템플릿, 그룹에서부터 VPC 네트워크, 방화벽 규칙, VPN 터널, Cloud Router, 부하 분산기에 이르기까지 우리가 지금까지 보았던 것의 모든 리소스를 배포할 수 있습니다. 지원되는 리소스 유형의 전체 목록은 [Google Cloud에서 Terraform 사용](#) 문서를 참고하세요.

Terraform 언어

- Terraform 언어는 리소스를 선언하기 위한 인터페이스입니다.
- 리소스는 인프라 객체입니다.
- 구성 파일은 리소스의 관리를 관장합니다.

```
resource "google_compute_network" "default" {
  name = "${var.network_name}"
  auto_create_subnetworks = false
}

<BLOCK TYPE> "<BLOCK LABEL>" "<BLOCK LABEL>" {
  # Block body
  <IDENTIFIER> = <EXPRESSION> # Argument
}
```

Terraform 언어는 리소스를 선언하기 위한 사용자 인터페이스입니다. 리소스는 **Compute Engine** 가상 머신, 스토리지 버킷, 컨테이너, 네트워크와 같은 인프라 객체입니다. Terraform 구성은 Terraform 언어로 된 완전한 문서로, Terraform에 주어진 인프라 컬렉션을 관리하는 방법을 알려줍니다. 하나의 구성은 여러 개의 파일과 디렉터리로 이루어질 수 있습니다.

Terraform 언어에는 다음과 같은 구문이 있습니다.

- 블록: 객체를 나타내며 0개 이상의 라벨을 가질 수 있습니다. 블록에는 인수와 중첩된 블록을 선언할 수 있는 본문이 있습니다.
- 인수: 이름에 값을 할당하는 데 사용됩니다.
- 표현식: 식별자에 할당할 수 있는 값을 나타냅니다.

여러 퍼블릭 및 프라이빗 클라우드에서 사용할 수 있는 Terraform

- Google Cloud의 최우선 도구로 간주됨
- Cloud Shell에 이미 설치되어 있음

```
provider "google" {  
  region = "us-central1"  
}  
  
resource "google_compute_instance" {  
  name         = "instance name"  
  machine_type = "n1-standard-1"  
  zone         = "us-central1-f"  
  
  disk {  
    image = "image to build instance"  
  }  
}  
  
output "instance_ip" {  
  value = "${google_compute_instance.ip_address}"  
}
```

Google Cloud

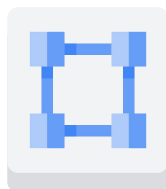
Terraform은 여러 퍼블릭 및 프라이빗 클라우드에서 사용할 수 있습니다. Terraform은 Cloud Shell에 이미 설치되어 있습니다.

슬라이드에 나와 있는 Terraform 구성 파일 예는 제공업체가 Google Cloud임을 나타내는 **provider** 블록으로 시작합니다. 배포할 리전은 **provider** 블록 내에 지정되어 있습니다.

resource 블록은 Google Cloud Compute Engine 인스턴스 또는 가상 머신을 지정합니다. 생성할 인스턴스의 세부정보는 **resource** 블록 내에 지정되어 있습니다.

output 블록은 Terraform 모듈의 출력 변수를 지정합니다. 여기서는 값이 출력 변수 'instance_ip'에 할당됩니다.

예: HTTP 방화벽 규칙을 갖는 자동 모드 네트워크



자동 모드
네트워크



HTTP 방화벽
규칙

Terraform으로 된 간단한 예를 살펴보겠습니다.

실습을 시작하기에 앞서, Terraform을 사용하여 하나의 HTTP 방화벽 규칙을 갖는 자동 모드 네트워크를 만드는 방법을 설명하겠습니다.

이 예에서는 인프라를 **main.tf**라는 하나의 파일 안에 정의합니다.

인프라의 복잡성이 증가하면 간편한 관리를 위해 각 요소를 개별적인 파일 안에 빌드할 수 있습니다.

예: HTTP 방화벽 규칙을 갖는 자동 모드 네트워크

```
main.tf
provider "google" {
}
```

main.tf 파일부터 시작하겠습니다. **main.tf** 파일은 생성할 인프라를 지정하는 곳입니다. 이 파일은 원하는 상태를 나타내는 청사진 역할을 합니다.

먼저 공급업체를 정의합니다.

예: HTTP 방화벽 규칙을 갖는 자동 모드 네트워크

```
main.tf
provider "google" {
}
# [START vpc_auto_create]
resource "google_compute_network" "vpc_network" {
  project      = var.project_id
  name         = "my-auto-mode-network"
  auto_create_subnetworks = true
  mtu          = 1460
}
```

다음으로 자동 생성 서브네트워크 플래그를 **true**로 설정하여 네트워크를 정의합니다. 이렇게 하면 자동으로 각 리전에 하나의 서브네트워크가 생성됩니다.

mtu는 1460으로 설정합니다.

예: HTTP 방화벽 규칙을 갖는 자동 모드 네트워크

```
main.tf
provider "google" {
}
.
.
.

# [START vpc_firewall_create]
resource "google_compute_firewall" "rules" {
  project      = var.project_id
  name         = "my-firewall-rule"
  network      = "vpc_network"
  allow {
    protocol = "tcp"
    ports    = ["80", "8080"]
  }
}
```

Google Cloud

다음으로, 방화벽을 정의합니다. TCP가 포트 80과 8080에 액세스할 수 있도록 허용하겠습니다.

Terraform은 main.tf 파일을 받아서 생성할 인프라의 사양으로 사용합니다.

Terraform으로 인프라 배포

```
terraform init
```

```
terraform plan
```

```
terraform apply
```

Google Cloud

main.tf 파일을 작성했으면 정의된 인프라를 **Cloud Shell**에 배포할 수 있습니다.

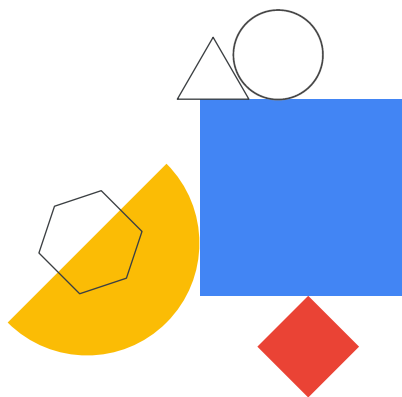
terraform init 명령어를 사용하여 새 **Terraform** 구성을 초기화합니다. 이 명령어는 **main.tf** 파일과 동일한 폴더에서 실행합니다. **terraform init** 명령어는 **Google** 제공업체 플러그인이 다운로드되어 현재 작업 디렉터리의 하위 디렉터리에 각종 관리 파일과 함께 설치되도록 합니다. 이 과정에서 '제공업체 플러그인 초기화 중' 메시지가 표시됩니다. **Terraform**은 실행이 **Google** 프로젝트에서 이루어지고 있음을 알고 있으며 **Google** 리소스를 얻고 있습니다.

terraform plan 명령어는 명시적으로 비활성화되지 않은 한 새로고침을 실행한 다음 구성 파일에 지정된 원하는 상태를 달성하려면 어떤 동작이 필요한지 알아냅니다. 이 명령어는 실제 리소스나 상태를 변경하지 않고도 변경사항 세트의 실행 계획이 예상과 일치하는지 확인해 볼 수 있는 편리한 방법입니다.

terraform apply 명령어는 **main.tf** 파일에 정의된 인프라를 생성합니다. 이 명령어가 완료되면 정의된 인프라에 액세스할 수 있습니다.

실습 소개

Terraform을 사용하여 인프라 배포
자동화

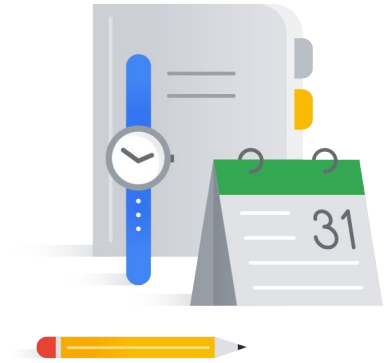


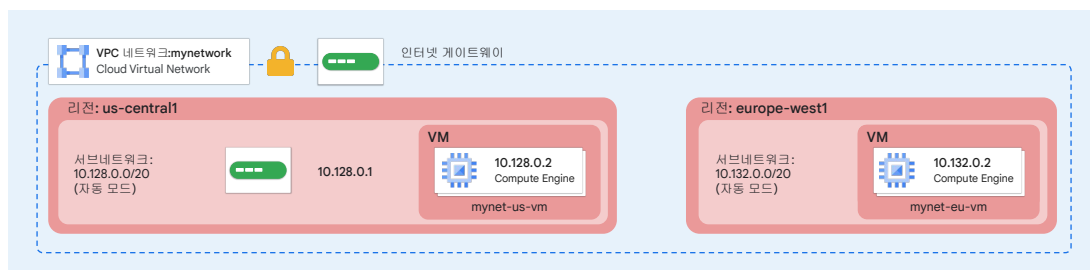
Google Cloud

이제까지 배운 내용을 적용하여 VPC 네트워크, 방화벽 규칙, VM 인스턴스의 배포를 자동화해 보겠습니다.

실습 목표

- 01 자동 모드 네트워크 구성 만들기
- 02 방화벽 규칙 구성 만들기
- 03 VM 인스턴스 템플릿 만들기
- 04 구성 만들기 및 배포하기
- 05 구성 배포 확인하기





HTTP, SSH, RDP, ICMP 트래픽을 허용하는 방화벽 규칙을 갖는 자동 모드 네트워크 배포합니다. 이 네트워크 다이어그램에 나와 있는 VM 인스턴스도 배포합니다.



Google Cloud Marketplace

Google Cloud

Google Cloud Marketplace를 조금 더 자세히 살펴보겠습니다.

Google Cloud Marketplace

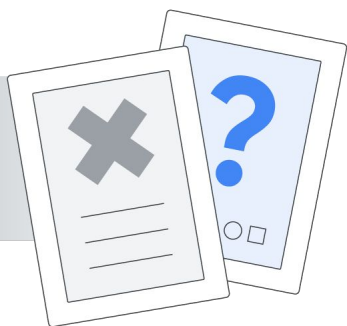
- 프로덕션 등급 솔루션 배포하기
- Google Cloud 서비스와 타사 서비스에 대한 단일 청구서
- Terraform을 사용하여 솔루션 관리하기
- 보안 업데이트가 제공되면 알림 전송
- 파트너에게 직접 지원받기

Google Cloud

Google Cloud Marketplace를 사용하면 Google Cloud에서 실행되는 소프트웨어 패키지를 빠르게 배포할 수 있습니다. Cloud Marketplace는 기존에 Terraform을 기반으로 자체 배포 구성을 생성한 타사 공급업체의 프로덕션 등급 솔루션을 제공합니다. 이러한 솔루션은 프로젝트의 모든 Google Cloud 서비스와 함께 청구됩니다. 타사 서비스 라이선스를 보유하고 있다면 Bring Your Own License(사용자 라이선스 사용) 솔루션의 사용이 가능할 수 있습니다.

일단 소프트웨어 패키지를 배포한 후 나중에 애플리케이션에 추가 용량이 필요하게 되면 배포를 확장할 수 있습니다. Google Cloud는 심각한 문제와 취약점을 수정하기 위해 소프트웨어 패키지의 이미지를 업데이트하지만, 이미 배포한 소프트웨어는 업데이트하지 않습니다. 파트너에게 직접 지원을 받을 수도 있습니다.

Cloud Marketplace에서 인프라 솔루션을 출시하는 방법을 알아보려면 이 [동영상](#)을 시청하세요.



퀴즈



질문 1

질문

Terraform 구성의 템플릿을 작성하면 어떤 이점이 있나요?

- A. 구성 관리 소프트웨어를 실행할 수 있습니다.
- B. 리소스의 속성을 하드코딩할 수 있습니다.
- C. 구성을 일부분의 재사용 가능한 개별 빌딩 블록으로 추상화할 수 있습니다.

질문 1

답변

Terraform 구성의 템플릿을 작성하면 어떤 이점이 있나요?

- A. 구성 관리 소프트웨어를 실행할 수 있습니다.
- B. 리소스의 속성을 하드코딩할 수 있습니다.
- C. 구성을 일부분의 재사용 가능한 개별 빌딩 블록으로 추상화할 수 있습니다.



Google Cloud

설명:

- A. 정답이 아닙니다. 프로비저닝 도구는 구성 관리 소프트웨어를 실행하는 데 사용됩니다. 템플릿 작성은 **Terraform** 구성 코드를 읽고 재사용할 수 있도록 만드는 작성 권장사항입니다.
- B. 정답이 아닙니다. 템플릿에서 템플릿 속성을 사용하여 템플릿을 유연하게 만들 수 있습니다.
- C. 정답입니다. 템플릿을 만든 후에는 필요에 따라 여러 배포에서 재사용할 수 있습니다. 마찬가지로 매우 비슷한 속성을 공유하는 구성을 재작성하는 경우 공유된 부분을 템플릿으로 추상화할 수 있습니다.

질문 2

질문

Cloud Marketplace에서 제공하는 것은 무엇인가요?

- A. 모든 Google Cloud 서비스 및 애플리케이션을 위한 일원화된 청구 플랫폼
- B. 기존에 Terraform을 기반으로 자체 배포 구성을 생성한 타사 공급업체의 프로덕션 등급 솔루션
- C. VM 인스턴스 거래 플랫폼

질문 2

답변

Cloud Marketplace에서 제공하는 것은 무엇인가요?

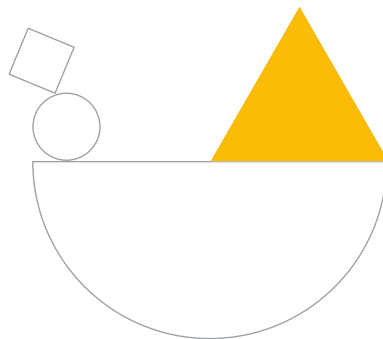
- A. 모든 Google Cloud 서비스 및 애플리케이션을 위한 일원화된 청구 플랫폼
- B. 기존에 **Terraform**을 기반으로 자체 배포 구성을 생성한 타사 공급업체의 프로덕션 등급 솔루션
- C. VM 인스턴스 거래 플랫폼



설명:

- A. 오답입니다. 과정 자료를 다시 살펴보세요.
- B. 정답입니다. **Cloud Marketplace**는 기존에 **Terraform**을 기반으로 자체 배포 구성을 생성한 타사 공급업체의 프로덕션 등급 솔루션을 제공합니다.
- C. 오답입니다. 과정 자료를 다시 살펴보세요.

리뷰: 인프라 자동화



Google Cloud

이 모듈에서는 **Terraform**을 사용하여 인프라의 배포를 자동화해 보고, **Cloud Marketplace**의 인프라 솔루션을 살펴보았습니다.

단지 네트워크 하나, 방화벽 규칙 한 개, 그리고 **VM** 인스턴스 두 개를 배포하기 위해 **Terraform**을 사용하는 것은 번거롭게 여겨질 수 있습니다. 이러한 리소스를 한 번만 생성하고 앞으로 다시는 생성할 계획이 없다면 번거로운 작업일 수 있겠지만, 다량의 리소스를 관리하고 반복 가능한 방식으로 배포, 업데이트, 폐기해야 하는 경우 **Terraform**과 같은 인프라 자동화 도구가 필요합니다.

